

File permissions in Linux

Project description

The research team is conducting a project to review and update file permissions within a specific directory. This includes checking the permissions of selected files and a subdirectory, then modifying them to match the organization's required authorization levels.

Check file and directory details

The project will be undertaken in the projects directory. The absolute file path to the directory is /home/researcher2/projects.

Step one involves navigating to the /home/researcher2/projects directory.

Step two involves displaying the contents of the directory and the permissions assigned to the files in the directory.

The following code demonstrates how I used Linux commands to do this:

```
researcher2@471452133e1d:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  3 13:29 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  3 13:49 ..
-rw--w---- 1 researcher2 research_team  46 Dec  3 13:29 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  3 13:29 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Dec  3 13:29 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  3 13:29 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  3 13:29 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  3 13:29 project_t.txt
researcher2@471452133e1d:~/projects$
```

The first line of the code sample is the input command, and the remaining lines show the output. The ls command lists the directory contents, while the -la options display file and subdirectory permissions. The a option ensures that hidden files are also shown

Description of permissions string

In Linux, file permissions are shown as a 10-character string such as drwxrwxrwx. The first character indicates the file type—d for a directory and - for a regular file. Characters 2–4 specify user permissions, 5–7 specify group permissions, and 8–10 specify permissions for others. The

letters r (read), w (write), and x (execute) show which permissions are granted; a hyphen (-) indicates a missing permission.

Hidden files and directories begin with a dot (.). For files, the dot is prefixed to the filename, and hidden directories are also represented with a leading dot

Change file permissions

To enhance system security and prevent unauthorized access, the organization decided that no files in the directory should allow write access for other users.

The first step was to check whether any files granted write permissions to others. One file was found to have this permission enabled. To meet the organization's security requirements and prevent unauthorized modifications, its permissions needed to be updated.

The following code demonstrates how I used Linux commands to do this:

```
researcher2@b107074c67e1:~/projects$ chmod o-w project_k.txt
researcher2@b107074c67e1:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  3 14:11 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  3 14:51 ..
-rw--w---- 1 researcher2 research_team  46 Dec  3 14:11 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  3 14:11 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Dec  3 14:11 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  3 14:11 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  3 14:11 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  3 14:11 project_t.txt
researcher2@b107074c67e1:~/projects$
```

The first line of the code sample is the command used to change the file's permissions. The second line displays the directory contents to confirm whether the change was applied, and the remaining lines show the output. The update was carried out using the symbolic method with the chmod command.

The file **project_k.txt** was found to have unauthorized write permissions for "other" users. In the chmod command, o refers to "others," the minus sign (-) removes a permission, and w specifies write access. The second argument identifies the file being modified.

The ls -la output confirmed that write permissions were successfully removed from **project_k.txt**, ensuring that no files in the directory allow write access for other users

Change file permissions on a restricted file

The file **project_m.txt** is classified as restricted, and only the user should have read and write permissions. Neither the group nor other users should have these permissions.

The first step is to check the file's current permissions using the `ls -la` command. The output below shows the permissions assigned to **project_m.txt**

```
researcher2@fcc09564ab25:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  3 15:29 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  3 15:51 ..
-rw--w---- 1 researcher2 research_team  46 Dec  3 15:29 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  3 15:29 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Dec  3 15:29 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  3 15:29 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  3 15:29 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  3 15:29 project_t.txt
researcher2@fcc09564ab25:~/projects$
```

The command output showed that the user had read and write permissions, the group also had read and write permissions, and others had no access. Because the group should not have read or write permissions, the current settings were non-compliant and required correction.

The next step was to update the file's permissions to match the required level of authorized access.

The below Linux code demonstrates the code used to change file permissions and the output thereof:

```
researcher2@fcc09564ab25:~/projects$ chmod g-r project_m.txt
researcher2@fcc09564ab25:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  3 15:29 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  3 15:51 ..
-rw--w---- 1 researcher2 research_team  46 Dec  3 15:29 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  3 15:29 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Dec  3 15:29 project_k.txt
-rw----- 1 researcher2 research_team  46 Dec  3 15:29 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  3 15:29 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  3 15:29 project_t.txt
researcher2@fcc09564ab25:~/projects$
```

The first line of the code sample applies the permission change, and the second line verifies the update by running `ls -la`. In the `chmod` command, the first argument specifies the permission adjustment: `g` refers to the group, the minus sign (`-`) removes a permission, and `r` indicates read access. The second argument is the filename to which the change is applied.

Change file permissions on a hidden file

In the projects directory, one file is identified as a hidden file. Hidden files are denoted by a leading period (.) in the file name. The hidden file in question is .projects_x.txt. The permissions currently assigned to this file do not comply with the level of authorization required by the organization

The permissions assigned are **read** and **write** for the **user** and **write** only for the **group**:

```
researcher2@8b55d1684975:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  4 10:25 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  4 11:47 ..
-rw--w---- 1 researcher2 research_team  46 Dec  4 10:25 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  4 10:25 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Dec  4 10:25 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  4 10:25 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  4 10:25 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  4 10:25 project_t.txt
researcher2@8b55d1684975:~/projects$
```

The organization has determined that the user and group should only have read permissions.

The following code demonstrates how I used Linux commands to do this:

```
researcher2@8b55d1684975:~/projects$ chmod u-w,g=r .project_x.txt
researcher2@8b55d1684975:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  4 10:25 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  4 11:47 ..
-r--r----- 1 researcher2 research_team  46 Dec  4 10:25 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  4 10:25 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Dec  4 10:25 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  4 10:25 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  4 10:25 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  4 10:25 project_t.txt
```

The first line contains the command used to modify file permissions by removing write access for the user and granting read-only access to the group. This was achieved using the symbolic mode of the chmod command. In symbolic notation:

- u-w removes write (w) permission from the user/owner (u).
- g=r sets the group (g) permission explicitly to read-only (r).

The symbolic method allows precise modification of individual permission bits without affecting other existing permissions.

Change directory permissions

This exercise demonstrates the group permissions on the drafts subdirectory. The absolute path to the directory is /home/researcher2/projects/drafts. According to organizational policy, the user researcher2 should have access exclusively to the drafts subdirectory. Additionally, this exercise illustrates how to modify permissions on the drafts subdirectory.

Step 1: Check the current permissions of the drafts subdirectory

The following code demonstrates how I used Linux commands to do this:

```
researcher2@8b55d1684975:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  4 10:25 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  4 11:47 ..
-r--r----- 1 researcher2 research_team  46 Dec  4 10:25 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  4 10:25 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Dec  4 10:25 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  4 10:25 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  4 10:25 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  4 10:25 project_t.txt
researcher2@8b55d1684975:~/projects$
```

The output of the command indicates that the group currently has execute (x) permissions. This constitutes unauthorized access and should be removed. The user, however, has read (r), write (w), and execute (x) permissions, which aligns with the level of authorization required by the organization.

Step 2: Modify the permissions for the group on the drafts subdirectory so that they comply with the organization's authorization policy. These permission changes will be performed using the symbolic method of the chmod command, employing the appropriate arguments to adjust the permission bits accordingly

The following code demonstrates how I used Linux commands to do this:

```
researcher2@502dfa01f9f0:~/projects$ chmod g-x drafts
researcher2@502dfa01f9f0:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  4 11:47 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  4 12:47 ..
-rw--w---- 1 researcher2 research_team  46 Dec  4 11:47 .project_x.txt
drwx----- 2 researcher2 research_team 4096 Dec  4 11:47 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Dec  4 11:47 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Dec  4 11:47 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  4 11:47 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Dec  4 11:47 project_t.txt
researcher2@502dfa01f9f0:~/projects$
```

The first line contains the command used to remove execute (x) permissions from the group for the draft's subdirectory. The second line of code verifies the changes by listing the contents of the projects directory. Note that the drafts directory is a subdirectory of projects. The output confirms that the user retains read (r), write (w), and execute (x) permissions on the draft's directory, which aligns with the required level of authorization. Additionally, the group has no read, write, or execute permissions, reflecting the correct level of authorization as mandated by the organization

Summary

This project demonstrated how to check permissions on files and subdirectories within the projects directory. It also illustrated how to modify permissions on specific files and subdirectories to align with the level of authorization required by the organization. All permission changes were implemented using the symbolic method in Linux, with commands designed to prevent unauthorized access and enhance the overall security of the file system